Guia QA - Skills de Automação do Cliente

Versão 1.0 - Abril 2026

1. Objetivo deste guia

Este guia ensina o QA a usar as skills de automação do BKPilot:

- `/plano-automacao` (consultoria de viabilidade — guia próprio em `Guia QA - Skill plano-automacao.md`)
- `/gerar-automacao-cliente`
- `/auditar-automacao-cliente`

O objetivo dessas skills é transformar o conhecimento gerado pelo pipeline de QA em um pacote de código de automação entregável ao cliente e, depois, auditar esse pacote antes que ele seja usado como base de trabalho por um QA automatizador.

Este guia também explica como acessar a VPS do BKPilot pelo Termius, como executar os comandos dentro da VPS e como baixar para o computador local os arquivos gerados pelas skills.

2. Para quem este guia foi escrito

Este guia foi escrito para o QA que vai operar o BKPilot em ambiente remoto, especialmente em uma VPS Linux.

Você não precisa ser especialista em infraestrutura, mas precisa seguir o fluxo com disciplina. Essas skills podem gerar código, logs internos de governança, relatórios e pacotes com artefatos de cliente. Por isso, pequenas improvisações podem gerar retrabalho ou vazamento de informação.

3. Visão geral das skills

3.1 `/gerar-automacao-cliente`

Esta skill gera um pacote de automação exportável no stack escolhido pelo cliente.

Ela usa como entrada os artefatos já produzidos pelo BKPilot, como:

- `estado/mapa.md`
- `estado/fluxos.md`
- `estado/elementos.json`

- `estado/api_endpoints.json`
- `cenarios/cenarios.xlsx`
- achados recentes em `resultado/latest/`, quando relevantes

Ela gera como saída uma pasta como:

```
entregaveis/<cliente>/automacao/<stack>/
```

Exemplo:

```
entregaveis/acme/automacao/playwright-ts/
```

Dentro dessa pasta ficam a especificação intermediária, o código gerado, a cobertura, pendências, inventario e o resumo da geração.

3.2 `/auditar-automacao-cliente`

Esta skill audita de forma independente o pacote gerado pela skill anterior.

Ela existe para impedir autoaprovação e para verificar se o código realmente reduz trabalho do QA automatizador. O foco da auditoria é encontrar:

- erro de sintaxe;
- import quebrado;
- método chamado mas não implementado;
- pseudo-código disfarçado de automação;
- seletor placeholder sem marcação clara;
- cobertura declarada como implementada, mas sem fluxo real;
- falta de assert útil;
- estrutura incoerente com o stack escolhido.

A auditoria gera, no pacote auditado:

```
auditoria_independente.md
```

Quando houver dado interno de governança, também pode gerar:

```
resultado/<timestamp>/governanca/auditoria_interna_<cliente>_<stack>.md
```

O arquivo interno não deve ser enviado ao cliente sem revisão, porque pode conter dados de governança.

4. Quando usar este fluxo

Use este fluxo quando o QA precisa entregar ao cliente uma base inicial de automação, por exemplo:

- uma suite Playwright em TypeScript;
- um projeto Playwright em JavaScript;
- uma base Cypress;
- uma base Python Pytest;
- um projeto Selenium Java;
- um projeto Selenium C#;
- uma suite Robot Framework.

Não use essas skills como substituto de exploracao, geração de cenários ou execução funcional. Elas dependem de uma base previa. Se a base estiver fraca, o código gerado também tende a ficar fraco.

5. Fluxo recomendado de ponta a ponta

O fluxo mais seguro e:

```
/plano-automacao
-> /explorar
-> /gerar-cenarios
-> /testar-modulo ou /executar-planilha
-> /gerar-automacao-cliente
-> /auditar-automacao-cliente
-> baixar pacote da VPS
-> revisar localmente
-> entregar ou encaminhar ao QA automatizador
```

A skill `/plano-automacao` deve ser executada primeiro para avaliar se o sistema está pronto para automação. As skills `/gerar-automacao-cliente` e `/auditar-automacao-cliente` deste guia entram somente depois que já existem artefatos suficientes para gerar automação.

6. Pre-requisitos na VPS

Antes de usar as skills, confirme que a VPS já está preparada:

- repositorio do BKPilot clonado;
- dependencias instaladas com `npm install`;
- Playwright e Chromium instalados;
- CLI operacional autenticada, normalmente Claude Code;
- arquivos `.env` configurados;

- artefatos de entrada já gerados;
- espaco em disco suficiente para criar pacote e compactar saidas.

Comandos rápidos de verificação:

```
pwd
ls
node -v
npm -v
npx playwright --version
claude --version
node converter/render.js --lint
```

Se `package.json`, `src/`, `.claude/commands/` e `documentacao_projeto/` não aparecerem no `ls`, você provavelmente não está na pasta correta do projeto.

7. Como baixar e instalar o Termius

Termius é um cliente SSH/SFTP usado para acessar a VPS e transferir arquivos.

Use sempre o site oficial:

```
https://termius.com/
```

Paginas oficiais úteis:

```
https://termius.com/download
https://www.termius.com/windows
https://termius.com/download/linux
https://termius.com/download/macos
```

7.1 Instalação no Windows

1. Acesse `https://www.termius.com/windows` ou `https://termius.com/download`. 2. Clique em `Download`. 3. Baixe o instalador `.exe`. 4. Execute o instalador. 5. Abra o Termius após a instalação. 6. Crie uma conta Termius se a equipe usar sincronização, ou use sem sincronizar se a política interna permitir.

7.2 Instalação no macOS

1. Acesse `https://termius.com/download/macos` ou `https://termius.com/download`. 2. Baixe o arquivo `.dmg`. 3. Abra o `.dmg`. 4. Arraste o Termius para `Applications`. 5. Abra o Termius.

7.3 Instalação no Linux local

1. Acesse `https://termius.com/download/linux`. 2. Baixe o pacote `.deb`, se estiver usando Ubuntu/Debian. 3. Instale pelo gerenciador de pacotes.

Exemplo:

```
sudo dpkg -i termius*.deb
sudo apt -f install
```

8. Dados que você precisa para acessar a VPS

Antes de configurar o Termius, tenha estes dados:

```
Host/IP: <IP_DA_VPS>
Porta SSH: 22
Usuario: <USUARIO_DA_VPS>
Senha ou chave SSH: <CREDENCIAL_FORNECIDA_PELO_RESPONSAVEL>
Pasta do projeto: /home/<USUARIO_DA_VPS>/bkpilot
```

Exemplo:

```
Host/IP: 203.0.113.10
Porta: 22
Usuario: bkpilot
Pasta: /home/bkpilot/bkpilot
```

Nunca envie senha da VPS ou chave SSH em chat, planilha ou comando colado em histórico publico.

9. Como criar o acesso da VPS no Termius

9.1 Criar um novo Host

1. Abra o Termius. 2. Va em `Hosts`. 3. Clique em `New Host`. 4. Preencha:

```
Label: BKPilot VPS
Address: <IP_DA_VPS>
Port: 22
Username: <USUARIO_DA_VPS>
```

5. Em autenticação, escolha uma das opcoes:

```
Password: se a VPS usa senha
Key: se a VPS usa chave SSH
```

6. Salve o host.

9.2 Conectar na VPS

1. Clique no host `BKPilot VPS`. 2. Aguarde abrir o terminal. 3. Se aparecer alerta de fingerprint, confirme apenas se o IP está correto e foi fornecido pelo responsavel da VPS. 4. Ao conectar, rode:

```
whoami
pwd
hostname
```

Esses comandos ajudam a confirmar se você entrou com o usuário esperado.

10. Entrar na pasta do BKPilot na VPS

Depois de conectar:

```
cd ~/bkpilot
pwd
ls
```

Se o projeto estiver em outro caminho:

```
cd /home/<USUARIO_DA_VPS>/bkpilot
```

O `ls` deve mostrar algo parecido com:

```
AGENTS.md
CLAUDE.md
README.md
package.json
src/
.claude/
estado/
resultado/
cenarios/
entregaveis/
```

Se não aparecer `package.json`, pare. Você está no diretório errado.

11. Conferir se os artefatos de entrada existem

Antes de gerar código, idealmente o sistema já passou por `/plano-automacao`, que validou a viabilidade técnica e o retorno do investimento.

A skill `/gerar-automacao-cliente` precisa de artefatos mínimos.

Verifique:

```
ls estado
ls cenarios
ls resultado
```

Arquivos esperados:

```
estado/mapa.md
estado/fluxos.md
estado/elementos.json
estado/api_endpoints.json
cenarios/cenarios.xlsx
```

Se algum desses arquivos não existir, execute antes:

```
/explorar <URL_DO_SISTEMA> --login <EMAIL_QA>
/gerar-cenarios --formato gherkin
```

Não gere automação sem base. Isso aumenta muito o risco de pacote bonito, mas inutil.

12. Abrir a CLI para executar as skills

Na VPS, dentro da pasta do projeto:

```
claude
```

Se a operação da equipe usa outra CLI, siga o padrão da equipe. Este guia usa Claude Code nos exemplos porque é o fluxo mais comum no projeto.

Depois que a CLI abrir, você podera digitar os slash commands.

13. Skill 1: `/gerar-automacao-cliente`

13.1 Para que serve

Use está skill para gerar um pacote inicial de automação no stack do cliente.

Ela não gera "qualquer código". Ela deve gerar um pacote auditavel, com:

- especificação intermediária;
- código-fonte no stack escolhido;
- mapeamento entre cenário e arquivo;
- cobertura implementada/parcial/não automatizada;
- pendências;
- inventario;
- auditoria técnica da própria geração;
- resumo executivo.

13.2 Sintaxe básica

```
/gerar-automacao-cliente <cliente> --stack <stack>
```

Exemplo:

```
/gerar-automacao-cliente acme --stack playwright-ts
```

13.3 Stacks suportados

```
playwright-ts
playwright-js
cypress-ts
python-pytest
selenium-java
selenium-csharp
robot-framework
```

13.4 Exemplos por stack

Playwright TypeScript:

```
/gerar-automacao-cliente acme --stack playwright-ts
```

Playwright JavaScript:

```
/gerar-automacao-cliente acme --stack playwright-js
```

Cypress TypeScript:

```
/gerar-automacao-cliente acme --stack cypress-ts
```

Python Pytest:

```
/gerar-automacao-cliente acme --stack python-pytest
```

Robot Framework:

```
/gerar-automacao-cliente acme --stack robot-framework
```

13.5 Gerar apenas um modulo

Use `--modulo` quando quiser gerar automação de uma área especifica.

Exemplo:

```
/gerar-automacao-cliente acme --stack playwright-ts --modulo Login
```

Use modulo quando:

- o projeto e grande;
- você quer validar a qualidade da saída antes de gerar tudo;
- o cliente pediu uma entrega parcial;
- o escopo de automação está dividido por dominio funcional.

13.6 Escolher padrão arquitetural

Use `--padrao` quando houver preferencia do cliente ou do time técnico.

Exemplo:

```
/gerar-automacao-cliente acme --stack playwright-ts --padrao page-objects
```

Opcoes esperadas:

```
page-objects
screenplay
keywords
```

Observação: o stack continua sendo soberano. Se o padrão solicitado conflitar com o stack, a skill deve adaptar e registrar a decisão.

13.7 Escolher idioma dos nomes e comentarios

Exemplo:

```
/gerar-automacao-cliente acme --stack playwright-ts --idioma en-US
```

Use `pt-BR` quando a equipe do cliente trabalha em portugues. Use `en-US` quando o cliente ou repositorio de automação trabalha em ingles.

13.8 Informar uma pasta de saída diferente

Por padrão, a saída fica em:

```
entregaveis/<cliente>/automacao/<stack>/
```

Se precisar mudar:

```
/gerar-automacao-cliente acme --stack playwright-ts --saida entrega-login/
```

Evite usar caminhos fora do projeto sem necessidade. Isso dificulta baixar, auditar e empacotar depois.

14. O que a geração deve criar

Depois da geração, confira:

```
ls entregaveis/<cliente>/automacao/<stack>/
```

Exemplo:

```
ls entregaveis/acme/automacao/playwright-ts/
```

Arquivos e pastas esperados:

```
especificacao_automacao.json
mapeamento_cenarios.md
codigo/
README_automacao.md
cobertura_automacao.md
pendencias.md
inventario_arquivos.md
auditoria_codigo.md
resumo_geracao.md
```

Também deve existir um log interno de autoria:

```
resultado/<timestamp>/governanca/automacao_autoria_<cliente>_<stack>.json
```

Esse arquivo é interno. Não envie ao cliente.

15. Como revisar rapidamente o pacote gerado

Antes de auditar formalmente, o QA deve fazer uma checagem rapida:

```
find entregaveis/<cliente>/automacao/<stack> -maxdepth 2 -type f | sort
```

Leia os principais arquivos:

```
sed -n '1,160p' entregaveis/<cliente>/automacao/<stack>/resumo_geracao.md
sed -n '1,160p' entregaveis/<cliente>/automacao/<stack>/cobertura_automacao.md
sed -n '1,160p' entregaveis/<cliente>/automacao/<stack>/pendencias.md
```

O QA deve procurar:

- cenário marcado como implementado sem arquivo correspondente;
- muitos `TODO` sem justificativa;
- código gerado em stack diferente do solicitado;
- ausência de `codigo/`;
- ausência de `auditoria_codigo.md`;
- cobertura prometendo mais do que foi gerado.

Se algo estiver claramente errado, registre e gere novamente com escopo menor ou com parametros mais precisos.

16. Skill 2: `/auditar-automacao-cliente`

16.1 Para que serve

Use está skill para auditar o pacote gerado por `/gerar-automacao-cliente`.

Ela é uma barreira de qualidade. O objetivo não e "passar pano" no código gerado. O objetivo é dizer se o pacote está:

```
aprovado
aprovado com ressalvas
reprovado
bloqueado por governanca
```

16.2 Regra mais importante

O mesmo agente/modelo/instancia que gerou o código não deve auditar o próprio código.

Se a segregacao não puder ser comprovada, a auditoria deve bloquear.

Isso existe para evitar autoaprovação.

16.3 Sintaxe básica

```
/auditar-automacao-cliente <cliente> --stack <stack>
```

Exemplo:

```
/auditar-automacao-cliente acme --stack playwright-ts
```

16.4 Auditar uma pasta especifica

Se a geração foi feita em pasta customizada:

```
/auditar-automacao-cliente acme --stack playwright-ts --pacote entrega-login/
```

16.5 Auditar um modulo especifico

```
/auditar-automacao-cliente acme --stack playwright-ts --modulo Login
```

16.6 Informar origem da geração

Se você tem o `geracao_id`, use `--origem`.

Exemplo:

```
/auditar-automacao-cliente acme --stack playwright-ts --origem auto-acme-1530
```

Use isso quando houver mais de uma geração para o mesmo cliente/stack e você quiser evitar que a skill audite o pacote errado.

17. O que a auditoria verifica

A auditoria deve verificar:

- se existe log interno de autoria;
- se o auditor e independente do gerador;
- se o pacote tem os artefatos obrigatorios;
- se o stack foi respeitado;
- se imports, requires, namespaces e caminhos fazem sentido;
- se os testes tem asserts úteis;
- se o código e executavel ou apenas plausível;
- se a cobertura declarada bate com os arquivos;
- se os cenários implementados realmente tem fluxo e validação.

Arquivos obrigatorios no pacote:

```
especificacao_automacao.json
mapeamento_cenarios.md
cobertura_automacao.md
inventario_arquivos.md
auditoria_codigo.md
codigo/
```

Se faltar algum, a auditoria deve parar como pacote incompleto.

18. Saidas da auditoria

Arquivo principal:

```
entregaveis/<cliente>/automacao/<stack>/auditoria_independente.md
```

Exemplo:

```
entregaveis/acme/automacao/playwright-ts/auditoria_independente.md
```

Esse é o arquivo que o QA pode revisar para entender o parecer técnico do pacote.

Arquivo interno, quando aplicavel:

```
resultado/<timestamp>/governanca/auditoria_interna_<cliente>_<stack>.md
```

Esse arquivo pode conter metadados internos de governança. Não envie ao cliente automaticamente.

19. Como interpretar o resultado

19.1 Aprovado

Use quando:

- não ha finding técnico relevante;
- stack está coerente;
- estrutura está funcional;
- cobertura declarada bate com código;
- validacoes obrigatorias foram executadas ou justificadas;
- não ha sinais de pseudo-código.

19.2 Aprovado com ressalvas

Use quando:

- o pacote e utilizavel;
- existem melhorias a fazer;
- ha duplicacao moderada;
- nomes poderiam melhorar;
- cobertura parcial está documentada;
- alguma validação não pode ser executada, mas a revisão manual não achou bloqueio.

19.3 Reprovado

Use quando existe qualquer problema que gere retrabalho imediato, por exemplo:

- erro de sintaxe;
- import quebrado;
- método inexistente;
- teste sem assert útil;
- estrutura de stack invalida;
- pseudo-código;

- cobertura enganosa.

19.4 Bloqueado

Use quando a auditoria não pode prosseguir por governança, principalmente:

- autoria não comprovada;
- auditor igual ao gerador;
- log interno ausente;
- `geracao_id` informado não encontrado;
- identidade do auditor atual não verificavel.

20. Como localizar os arquivos gerados na VPS

20.1 Ver entregaveis de um cliente

```
find entregaveis/<cliente> -maxdepth 4 -type f | sort
```

Exemplo:

```
find entregaveis/acme -maxdepth 4 -type f | sort
```

20.2 Ver somente o pacote de automação

```
find entregaveis/<cliente>/automacao/<stack> -maxdepth 4 -type f | sort
```

Exemplo:

```
find entregaveis/acme/automacao/playwright-ts -maxdepth 4 -type f | sort
```

20.3 Ver logs internos de governança

```
find resultado -path '*/governanca/*' -type f | sort
```

Lembrete: governança é interna. Baixe se precisar arquivar internamente, mas não envie ao cliente sem revisão.

21. Compactar o pacote antes de baixar

O jeito mais seguro de retirar muitos arquivos da VPS é compactar primeiro.

21.1 Criar pasta de exportação

```
mkdir -p exports
```

21.2 Compactar o pacote gerado

```
tar -czf exports/<cliente>_<stack>_automacao.tar.gz \
  entregaveis/<cliente>/automacao/<stack>
```

Exemplo:

```
tar -czf exports/acme_playwright-ts_automacao.tar.gz \
  entregaveis/acme/automacao/playwright-ts
```

21.3 Compactar também governança interna, se necessário

Use somente para arquivo interno BugKillers:

```
tar -czf exports/acme_playwright-ts_governanca.tar.gz resultado/*/governanca
```

Se houver dados de vários clientes em `resultado/`, prefira filtrar manualmente antes de compactar.

21.4 Conferir tamanho do arquivo

```
ls -lh exports
```

Se o arquivo estiver muito grande, verifique se vídeos ou screenshots foram incluídos sem necessidade.

22. Baixar os arquivos pelo Termius usando SFTP

Termius possui recurso de SFTP para navegar nos arquivos da VPS e baixar para o computador local.

22.1 Abrir SFTP

1. No Termius, conecte no host `BKPilot VPS`. 2. Abra a área de `SFTP` ou `Files`. 3. Navegue ate a pasta do projeto.

Caminho comum:

```
/home/<USUARIO_DA_VPS>/bkpilot
```

22.2 Acessar a pasta de exports

Entre em:

```
/home/<USUARIO_DA_VPS>/bkpilot/exports
```

Baixe o arquivo:

```
<cliente>_<stack>_automacao.tar.gz
```

Exemplo:

```
acme_playwright-ts_automacao.tar.gz
```

22.3 Onde salvar no computador local

Crie uma pasta local organizada:

```
BKPilot/
  entregas/
    acme/
      2026-04-24/
```

Evite salvar pacotes de clientes na área de downloads sem organização.

22.4 Descompactar no computador local

No Windows, você pode usar 7-Zip, WinRAR ou o próprio suporte do Windows dependendo da versão.

No macOS/Linux:

```
tar -xzf acme_playwright-ts_automacao.tar.gz
```

Depois confira:

```
find entregaveis/acme/automacao/playwright-ts -maxdepth 3 -type f | sort
```

23. Baixar por SCP, se preferir terminal local

Se você preferir baixar pelo terminal do seu computador local:

```
ARQUIVO="/home/<USUARIO_DA_VPS>/bkpilot/exports/"
ARQUIVO="${ARQUIVO}acme_playwright-ts_automacao.tar.gz"
scp <USUARIO_DA_VPS>@<IP_DA_VPS>:"$ARQUIVO" .
```

Exemplo:

```
ARQUIVO="/home/bkpilot/bkpilot/exports/"
ARQUIVO="${ARQUIVO}acme_playwright-ts_automacao.tar.gz"
scp bkpilot@203.0.113.10:"$ARQUIVO" .
```

Use SCP quando:

- o SFTP do Termius estiver lento;
- você já está confortável com terminal;
- precisa automatizar download de artefatos.

24. Baixar apenas alguns arquivos

Se você não precisa baixar o pacote inteiro, baixe apenas:

```
README_automacao.md
cobertura_automacao.md
pendencias.md
auditoria_independente.md
resumo_geracao.md
```

Mas para entrega técnica de automação, normalmente o pacote inteiro e necessário, porque o código fica em:

```
codigo/
```

25. O que pode e o que não pode ser enviado ao cliente

25.1 Pode ser enviado, após revisão

Normalmente pode ir para o cliente:

```
entregaveis/<cliente>/automacao/<stack>/codigo/
entregaveis/<cliente>/automacao/<stack>/especificacao_automacao.json
entregaveis/<cliente>/automacao/<stack>/mapeamento_cenarios.md
entregaveis/<cliente>/automacao/<stack>/README_automacao.md
entregaveis/<cliente>/automacao/<stack>/cobertura_automacao.md
entregaveis/<cliente>/automacao/<stack>/pendencias.md
entregaveis/<cliente>/automacao/<stack>/inventario_arquivos.md
entregaveis/<cliente>/automacao/<stack>/auditoria_codigo.md
entregaveis/<cliente>/automacao/<stack>/auditoria_independente.md
entregaveis/<cliente>/automacao/<stack>/correcoes_auditoria.md
entregaveis/<cliente>/automacao/<stack>/resumo_geracao.md
```

`correcoes_auditoria.md` só existirá quando a auditoria aplicar correções ou registrar defeitos não corrigíveis. Se ele existir, deve ser revisado e enviado junto com o pacote, porque explica exatamente o que foi corrigido antes da entrega.

25.2 Regra obrigatória de PDF para relatórios `.md`

Todo arquivo `.md` destinado ao cliente deve ser entregue também em PDF.

Regra para a IA/agente que executar estas skills:

- se gerar `README_automacao.md`, gerar também `README_automacao.pdf`;
- se gerar `mapeamento_cenarios.md`, gerar também `mapeamento_cenarios.pdf`;
- se gerar `cobertura_automacao.md`, gerar também `cobertura_automacao.pdf`;
- se gerar `pendencias.md`, gerar também `pendencias.pdf`;
- se gerar `inventario_arquivos.md`, gerar também `inventario_arquivos.pdf`;
- se gerar `auditoria_codigo.md`, gerar também `auditoria_codigo.pdf`;
- se gerar `auditoria_independente.md`, gerar também `auditoria_independente.pdf`;
- se gerar `correcoes_auditoria.md`, gerar também `correcoes_auditoria.pdf`;
- se gerar `resumo_geracao.md`, gerar também `resumo_geracao.pdf`.

Exemplo de conversão usando o conversor do projeto:

```
node cenarios/_md_to_pdf.js \
  entregaveis/acme/automacao/playwright-ts/auditoria_independente.md \
  entregaveis/acme/automacao/playwright-ts/auditoria_independente.pdf
```

Se houver muitos `.md`, gerar todos antes de compactar o pacote. O QA não deve entregar ao cliente um relatório `.md` sem o PDF correspondente.

25.3 Não enviar sem revisão interna

Não envie automaticamente:

```
resultado/<timestamp>/governanca/
automacao_autoria_<cliente>_<stack>.json
auditoria_interna_<cliente>_<stack>.md
.env
clients/<cliente>/.env
```

Motivo: esses arquivos podem conter metadados internos, referências de executor, tokens, caminhos sensíveis ou dados de governança.

26. Checklist antes de entregar o pacote

Antes de baixar ou enviar o pacote, confirme:

- [ ] `/plano-automacao` foi executada e o veredito foi "Viável" ou "Parcialmente Viável";
- [ ] `/gerar-automacao-cliente` terminou sem bloqueio;
- [ ] `resumo_geracao.md` existe;
- [ ] `codigo/` existe;
- [ ] `cobertura_automacao.md` existe;
- [ ] `pendencias.md` existe;
- [ ] `inventario_arquivos.md` existe;
- [ ] `auditoria_codigo.md` existe;
- [ ] `/auditar-automacao-cliente` foi executada por auditor independente;
- [ ] `auditoria_independente.md` existe;
- [ ] `correcoes_auditoria.md` foi revisado, se existir;
- [ ] todo relatório `.md` destinado ao cliente possui PDF correspondente;
- [ ] não ha `geracao_id`, modelo, agente ou executor em artefato destinado ao cliente;
- [ ] o pacote foi compactado em `exports/`;
- [ ] o arquivo compactado foi baixado pelo Termius/SFTP ou SCP;
- [ ] o pacote foi descompactado e revisado localmente.

27. Problemas comuns

27.1 Estou na VPS, mas não acho o projeto

Rode:

```
pwd
ls
find ~ -maxdepth 3 -name package.json 2>/dev/null
```

Entre na pasta correta:

```
cd /home/<USUARIO_DA_VPS>/bkpilot
```

27.2 A skill diz que faltam artefatos

Verifique:

```
ls estado
ls cenarios
```

Se faltar `estado/mapa.md` ou `cenarios/cenarios.xlsx`, rode antes `/explorar` e `/gerar-cenarios`.

27.3 O pacote foi gerado, mas a auditoria bloqueou

Leia:

```
find resultado -path '*/governanca/*' -type f | sort
```

Possíveis causas:

- log de autoria ausente;
- auditor igual ao gerador;
- `geracao_id` errado;
- identidade do auditor não verificavel.

Não force aprovação manual sem registrar o motivo. O bloqueio existe para evitar autoaprovação.

27.4 O Termius conecta, mas SFTP não abre

Verifique:

- se a conexão SSH está ativa;
- se o usuário tem permissão de leitura na pasta;
- se o caminho digitado existe;
- se você está usando o usuário correto.

Teste no terminal:

```
ls -la /home/<USUARIO_DA_VPS>/bkpilot
ls -la /home/<USUARIO_DA_VPS>/bkpilot/exports
```

27.5 O arquivo compactado ficou grande demais

Veja tamanho por pasta:

```
du -sh entregaveis/<cliente>/automacao/<stack>/*
du -sh resultado/*
```

Se necessário, compacte apenas o pacote de automação e deixe vídeos/logs de execução fora do download.

27.6 Baixei o pacote, mas ele veio vazio ou incompleto

Na VPS, confira o conteúdo antes de compactar:

```
find entregaveis/<cliente>/automacao/<stack> -maxdepth 4 -type f | sort
```

Recrie o arquivo:

```
rm -f exports/<cliente>_<stack>_automacao.tar.gz
tar -czf exports/<cliente>_<stack>_automacao.tar.gz \
  entregaveis/<cliente>/automacao/<stack>
ls -lh exports/<cliente>_<stack>_automacao.tar.gz
```

28. Exemplo completo de operação

Contexto

```
Cliente: acme
Stack: playwright-ts
VPS: 203.0.113.10
Usuario VPS: bkpilot
Projeto na VPS: /home/bkpilot/bkpilot
```

Passo 1: conectar pelo Termius

Criar host:

```
Label: BKPilot VPS
Address: 203.0.113.10
Port: 22
Username: bkpilot
```

Conectar e entrar no projeto:

```
cd /home/bkpilot/bkpilot
ls
```

Passo 2: abrir Claude Code

```
claude
```

Passo 3: gerar automação

Dentro da CLI:

```
/gerar-automacao-cliente acme --stack playwright-ts --modulo Login
```

Passo 4: conferir pacote

No terminal da VPS:

```
find entregaveis/acme/automacao/playwright-ts -maxdepth 4 -type f | sort
```

Passo 5: auditar pacote

Dentro da CLI, idealmente em executor independente:

```
/auditar-automacao-cliente acme --stack playwright-ts --modulo Login
```

Passo 6: ler parecer

```
sed -n '1,200p' \
  entregaveis/acme/automacao/playwright-ts/auditoria_independente.md
```

Passo 7: compactar

```
mkdir -p exports
tar -czf exports/acme_playwright-ts_automacao.tar.gz \
  entregaveis/acme/automacao/playwright-ts
ls -lh exports
```

Passo 8: baixar pelo Termius

No SFTP do Termius:

```
/home/bkpilot/bkpilot/exports/acme_playwright-ts_automacao.tar.gz
```

Baixe para:

```
BKPilot/entregas/acme/2026-04-24/
```

Passo 9: revisar localmente

Descompacte e confira:

```
tar -xzf acme_playwright-ts_automacao.tar.gz
find entregaveis/acme/automacao/playwright-ts -maxdepth 3 -type f | sort
```

29. Boas práticas de segurança

- Nunca coloque senha no comando.
- Nunca baixe `.env`.
- Nunca envie `resultado/<timestamp>/governanca/` ao cliente sem revisão.
- Nunca compartilhe chave SSH em chat.
- Nunca compacte a raiz inteira do projeto para enviar ao cliente.
- Sempre revise `auditoria_independente.md` antes de entregar.
- Sempre confirme se o pacote não contem identidade de agente, modelo ou executor.
- Sempre remova exports antigos se a VPS estiver ficando cheia.

Para limpar exports antigos, depois de confirmar que já foram arquivados:

```
ls -lh exports
rm exports/<arquivo_antigo>.tar.gz
```

Use `rm` com cuidado. Confira o nome do arquivo antes de apagar.

30. Resumo operacional rápido

Comandos principais:

```
/gerar-automacao-cliente acme --stack playwright-ts
/auditar-automacao-cliente acme --stack playwright-ts
```

Pasta principal:

```
entregaveis/acme/automacao/playwright-ts/
```

Compactar:

```
mkdir -p exports
tar -czf exports/acme_playwright-ts_automacao.tar.gz \
  entregaveis/acme/automacao/playwright-ts
```

Baixar pelo Termius:

```
SFTP -> /home/<USUARIO_DA_VPS>/bkpilot/exports/
```

O arquivo mais importante da auditoria:

```
auditoria_independente.md
```

O arquivo que não deve ser enviado automaticamente:

```
resultado/<timestamp>/governanca/auditoria_interna_<cliente>_<stack>.md
```

31. Fontes externas consultadas

- Termius site oficial: `https://termius.com/`
- Download Termius: `https://termius.com/download`
- Termius para Windows: `https://www.termius.com/windows`
- Termius para Linux: `https://termius.com/download/linux`
- Termius para macOS: `https://termius.com/download/macos`